LAPORAN PRAKTIKUM STRUKTUR DATA

ABSTRACT DATA TYPE (ADT) JAM

Disusun Oleh :

Aliifah Felda Mufarrihati Salwaa

2511531011

Dosen Pengampu :

Dr. Wahyudi, M.T

Asisten Praktikum :

M Galid Avaro

DEPARTEMEN INFORMATIKA

FAKULTAS TEKNOLOGI INFORMASI

UNIVERSITAS ANDALAS

TAHUN 2026

KATA PENGANTAR

Laporan praktikum ini disusun sebagai salah satu bentuk pertanggungjawaban kegiatan praktikum Struktur Data yang membahas tentang Abstract Data Type (ADT) Jam. Melalui laporan ini penulis dapat memahami materi praktikum secara mendalam dan dapat lebih teliti, teratur, serta memiliki kemampuan yang baik dalam penulisan kode program sesuai kaidah akademik. Sehingga laporan ini dapat menjadi sarana belajar, dokumentasi kegiatan, dan referensi praktikum berikutnya.

Penulis menyadari bahwa laporan ini masih memiliki banyak kekurangan, baik dari isi maupun penyajiannya. Oleh karena itu, saran dan kritik sangat penulis harapkan guna untuk menyempurnakan laporan berikutnya.

Padang, 01 April 2026

Aliifah Felda Mufarrihati Salwaa
2511531011

DAFTAR ISI

KATA PENGANTAR i

BAB I PENDAHULUAN 3

1.1 Latar Belakang Praktikum 3

1.2 Tujuan Praktikum 3

1.3 Manfaat Praktikum 3

BAB II PEMBAHASAN 4

2.1. Dasar Teori 4

2.2. Abstract Data Type (ADT) JAM dalam bahasa algoritmik dan Penerapannya dalam bahasa pemrograman Java 4

2.3. Penggunaan Class jam dengan jamDriver 9

2.4. Tugas Praktikum Pekan 1 – ADT Mobil dan Drivernya 12

BAB III KESIMPULAN 16

3.1. Kesimpulan 16

3.2. Saran 16

DAFTAR PUSTAKA 17

BAB I

PENDAHULUAN

1.1 Latar Belakang Praktikum

Laporan praktikum ini disusun sebagai salah satu bentuk pemahaman terhadap materi Abstract Data Type (ADT) yang diterapkan pada objek jam dan mobil. ADT merupakan konsep dasar dalam struktur data yang memungkinkan suatu tipe data didefinisikan berdasarkan operasi-operasi yang dapat dilakukan tanpa memperhatikan implementasi internalnya.

Pada praktikum ini, dilakukan pembuatan ADT jam dengan atribut jam, menit, detik, serta metode selektor dan mutator. Melalui penerapan ini, praktikan dapat memahami bagaimana konsep ADT diimplementasikan dalam bahasa pemrograman Java secara terstruktur dan sistematis.

Selain itu, penggunaan class driver juga membantu dalam memahami bagaimana suatu ADT digunakan dalam program nyata. Dengan demikian, praktikum ini menjadi sarana untuk melatih kemampuan dalam merancang program berbasis objek serta meningkatkan pemahaman terhadap konsep dasar struktur data.

1.2 Tujuan Praktikum

Adapun tujuan dari praktikum ini adalah sebagai berikut:

1. Memahami konsep Abstract Data Type (ADT) dalam pemrograman.
2. Mengimplementasikan ADT mobil menggunakan bahasa pemrograman Java.
3. Melatih kemampuan dalam membuat program yang terstruktur dan sesuai dengan konsep OOP.

1.3 Manfaat Praktikum

Manfaat yang diperoleh dari praktikum ini adalah sebagai berikut:

1. Memahami konsep ADT secara lebih mendalam.
2. Menjadi dasar dalam mempelajari struktur data yang lebih kompleks.
3. Praktikan mampu mengimplementasikan ADT dalam bentuk program Java.

BAB II
PEMBAHASAN

2.1. Dasar Teori

Abstract Data Type (ADT) merupakan suatu definisi abstrak untuk operasi dan sifat yang diinginkan dari sebuah program. ADT adalah sistem dimana suatu program didefinisikan hanya melalui input dan output tanpa perlu mengetahui proses atau implementasi internal yang dilakukan.

Fungsi/Prosedur ADT:

- Konstruktor/Kreator
- Selektor (getter) : untuk mengakses komponen type
- Mutator (setter) : untuk mengubah nilai komponen type
- Validator : untuk uji pembentukan type
- Destruktor/Dealokator : untuk menghancurkan nilai objek sekaligus memori penyimpanannya
- Baca/Tulis : untuk interface dengan input/output device
- Operator relational : mendefinisikan hubungan relasional
- Operator aritmatika : mendefinisikan operasi aritmatika
- Konversi type

2.2. Abstract Data Type (ADT) JAM dalam bahasa algoritmik dan Penerapannya dalam bahasa pemrograman Java

Pada class ini kita tidak akan bisa melakukan run program, karena class ini hanya sebagai ADT tempat dimana kita melakukan inisialisai tipe data dan fungsi yang akan digunakan pada class driver nantinya.

Mendefinisikan tipe data:

```
{ *** Definisi TYPE JAM *** }
type Hour : integer [0..23]
type Minute : integer [0..59]
```

```
type Second : integer [0..59]

type JAM : < HH : Hour, { Hour [0..23] } MM : Minute, { Minute [0..59] } SS :
Second { Second [0..59] } >
{ ************************************************************ }
```

Tabel 2.1: definisi tipe data

```
package pekan1_2511531011;

public final class jam_2511531011 {
	private int hh; //0..23
	private int mm; //0..59
	private int ss; //00.59
```

Kode Program 2.1: definisi tipe data

Fungsi untuk validasi data dan membuat sebuah jam dari komponen-komponennya:

```
{ DEFINISI PRIMITIF }
{ ************************************************************ }
{ KELOMPOK VALIDASI TERHADAP TYPE }
{ ************************************************************ }
function IsJValid (H, M, S : integer) → boolean
{ Mengirim true jika H,M,S dapat membentuk J yang valid }
{ dipakai untuk mentest SEBELUM membentuk sebuah Jam }
{ ** Konstruktor: Membentuk sebuah JAM dari komponen-komponennya ** }
function MakeJam (HH : integer, MM : integer, SS : integer) → JAM
{ Membentuk sebuah JAM dari komponen-komponennya yang valid }
{ Prekondisi : HH, MM, SS valid untuk membentuk JAM }
```

Tabel 2.2: validasi dan konstruktor

```
	// Validator
	public static boolean isValid(int h, int m, int s) {
		return (0 <=h && h <= 23) && (0 <=m && m <= 59) && (0 <=s
&& s <= 59);
	}

	// Konstruktor
	public jam_2511531011(int h, int m, int s) {
		this.hh = h; this.mm = m; this.ss = s;
	}
```

Tabel 2.2: validasi dan konstruktor

Fungsi getter (selektor) untuk mengambil nilai data dan prosedur setter (mutator) untuk mengubah nilai data:

```
{ *** Operasi terhadap komponen : selektor Get dan Set *** }
{ *** Selektor *** }
function GetHour (J : JAM) → integer
{ Mengirimkan bagian HH (Hour) dari JAM }
function GetMinute (J : JAM) → integer
{ Mengirimkan bagian Menit (MM) dari JAM }
function GetSecond (J : JAM) → integer
{ Mengirimkan bagian Second (SS) dari JAM }
{ *** Pengubah nilai Komponen *** }
procedure SetHH (input/output J : JAM, input newHH : integer)
{ Mengubah nilai komponen HH dari J }
procedure SetMM (input/output J : JAM, input newMM : integer)
{ Mengubah nilai komponen MM dari J }
procedure SetSS (input/output J : JAM, input newSS : integer)
{ Mengubah nilai komponen SS dari J }
{ *** Destruktor *** }
{ Tidak perlu. Akan dijelaskan kemudian. }
{ ********************************************************** }
```

Tabel 2.3: getter setter

```
// Selektor
public int getHour() { return hh; }
public int getMinute() { return mm; }
public int getSecond() { return ss; }

// Mutator
public void setHour(int h) { this.hh = h; }
public void setMinute(int m) { this.mm = m; }
public void setSecond(int s) { this.ss = s; }
```

kode Program 2.3: getter setter

Fungsi untuk konversi data:

```
{ KELOMPOK KONVERSI TERHADAP TYPE }
{ ********************************************************** }
function JamToDetik (J : JAM) → integer
{ Diberikan sebuah JAM, mengkonversi menjadi Detik }
{ Rumus : detik = 3600*hour+menit*60 + detik  }
{ nilai maksimum = 3600*23+59*60+59*60 }
{ hati-hati dengan representasi integer pada bahasa implementasi }
{ kebanyakan sistem mengimplementasi integer, }
{ bernilai maksimum kurang dari nilai maksimum hasil konversi }
function DetikToJam (N : integer) → JAM
{ Mengirim  konversi detik ke JAM }
{ pada beberapa bahasa, representasi integer tidak cukup untuk }
```

```
{ menampung N }
{ ************************************************************ }
```

Tabel 2.4: konversi data

```
        // Konversi
        public int toSeconds() { return hh * 3600 + mm * 60 + ss; }
        public static jam_2511531011 fromSeconds(int total) {
                if (total < 0) throw new IllegalArgumentException("detiknegatif");
                total %= 24 * 3600; //wrap 24 jam
                int h = total / 3600; total %= 3600;
                int m = total / 60; int s =  total % 60;
                return new jam_2511531011(h, m, s);
        }
```

Kode Program 2.4: konversi data

Fungsi untuk menjalankan operasi relasional terhadap data:

```
{ KELOMPOK OPERASI TERHADAP TYPE }
{ ************************************************************ }
{ *** Kelompok Operator Relational *** }
function JEQ (J1 : JAM, J2 : JAM) → boolean
{ Mengirimkan true jika J1=J2, false jika tidak }
function JNEQ (J1 : JAM, J2 : JAM) → boolean
{ Mengirimkan true jika J1 tidak sama dengan J2 }
function JLT (J1 : JAM, J2 : JAM) → boolean
{ Mengirimkan true jika J1<J2 , false jika tidak }
function JGT (J1 : JAM, J2 : JAM) → boolean
{ Mengirimkan true jika J1>J2, false jika tidak }
```

Tabel 2.5: operasi relasional

```
        // Relasional
        public int compareTo(jam_2511531011 other) {
                return Integer.compare(this.toSeconds(), other.toSeconds());
        }
        public boolean equals(Object o) {
                if (!(o instanceof jam_2511531011 j)) return false;
                return hh == j.hh && mm == j.mm && ss == j.ss;
        }
        public int hashCode( ) {
                return java.util.Objects.hash(hh, mm, ss);
        }
```

Kode Program 2.5: operasi relasional

Fungsi untuk melakukan operasi aritmatika terhadap data:

```
 *** Operator aritmatika JAM *** }
function JPlus (J1 : JAM, J2 : JAM) → JAM
{ Menghasilkan J1+J2, dalam bentuk JAM }
function JMinus (J1 : JAM, J2 : JAM) → JAM
{ Menghasilkan J1-J2, dalam bentuk JAM }
{ Prekondisi : J1<=J2 }
function NextDetik (J : JAM) → JAM
{ Mengirim 1 detik setelah J dalam bentuk JAM  }
function NextNDetik (J : JAM, N : integer) → JAM
{ Mengirim N detik setelah J dalam bentuk JAM }
function PrevDetik (J : JAM) → JAM
{ Mengirim 1 detik sebelum J dalam bentuk JAM }
function PrevNDetik (J : JAM, N : integer) → JAM
{ Mengirim N detik sebelum J dalam bentuk JAM }
```

Tabel 2.6: operasi aritmatika

```
        // Aritmatika
        public jam_2511531011 plus (jam_2511531011 other ) {
                return fromSeconds(this.toSeconds() + other.toSeconds());
        }
        public jam_2511531011 minus (jam_2511531011 other ) {
                return fromSeconds(Math.floorMod(this.toSeconds() -
other.toSeconds(), 24*3600));
        }
        public jam_2511531011 nextSecond() {
                return fromSeconds(this.toSeconds() + 1);
        }
        public jam_2511531011 nextNSecond(int n) {
                return fromSeconds(this.toSeconds() + Math.max(0,  n));
        }
        public jam_2511531011 prevSecond() {
                return fromSeconds(Math.floorMod(this.toSeconds() - 1,
24*3600));
        }
        public jam_2511531011 prevNSecond(int n) {
                return fromSeconds(Math.floorMod(this.toSeconds() - Math.max(0,
n), 24*3600));
        }
```

Kode Program 2.6: operasi aritmatika

```
{ *** Kelompok Operator Aritmetika *** }
function Durasi (Jaw : JAM , Jakh : JAM) → integer
```

```
{ Mengirim JAkh -JAw dlm Detik, dengan kalkulasi }
{ Hasilnya negatif jika Jaw > JAkhir }
{ ************************************************************* }
{ Dapat ditambahkan fungsi lain }
```

Kode Program 2.7: operasi aritmatika lanjutan

```
	// Durasi (detik) bisa negatif jika this > other? sesuai spesifikasi durasi
(JAw, JAkh)
	public static int durasiDetik(jam_2511531011 jaw, jam_2511531011 jakh)
{
		return jakh.toSeconds() - jaw.toSeconds();
	}
	// Overriding ke String
	public String toString() {
		return String.format("%02d:%02d:%02d", hh, mm, ss);
	}

}
```

Kode Program 2.7: operasi aritmatika lanjutan

2.3. Penggunaan Class jam dengan jamDriver

a. jamDriver_2511531011

Pada class jamDriver_2511531011 ini kita akan menggunakan seluruh tipe data yang telah kita insisialisasi pada class jam_251131011. Disini kita dapat menggunakan fungsi dan prosedur yang telah dibangun, serta input data langsung dalam kode program.

```
package pekan1_2511531011;

public class jamDriver_2511531011 {

	public static void main(String[] args) {
		jam_2511531011 a = new jam_2511531011(23,59,50);
		jam_2511531011 b = new jam_2511531011(0, 0, 15);
		System.out.println("a		= " + a);
		System.out.println("b		= " + b);
		System.out.println("a+b		= " + a.plus(b));
		System.out.println("next 20s	= " + a.nextNSecond(20));
		System.out.println("durasi(a,b)	= " +
jam_2511531011.durasiDetik(a, b));
		System.out.println("a.compareTo(b)	= " + a.compareTo(b));


	}
```

```
}
```

Kode Program 2.8: Driver jam

Output program:

```
a                 = 23:59:50
b                 = 00:00:15
a+b               = 00:00:05
next 20s          = 00:00:10
durasi(a,b)       = -86375
a.compareTo(b)    = 1
```

Gambar 2.1: Output Driver jam

b. Class jamDriver2_2511531011

Pada class jamDriver2_2511531011 kita akan menggunakan fungsi dan prosedur yang terdapat pada class jam_2511531011 dan melakukan input langsung dari user (pada output program).

```
package pekan1_2511531011;

import java.util.Scanner;

public class jamDriver2_2511531011 {

        public static void main(String[] args) {
                Scanner input = new Scanner(System.in);
                System.out.println("=== Program Driver Objek Jam ===");
                // 1. Input Jam Pertama
                System.out.println("\n--- Input Jam 1 ---");
                jam_2511531011 j1 = buatJamDariInput(input);

                // 2. Input Jam Kedua
                System.out.println("\n--- Input Jam 2 ---");
                jam_2511531011 j2 = buatJamDariInput(input);

                // 3. Menampilkan data
                System.out.println("\n--- Hasil Operasi ---");
                System.out.println("Jam 1 (String)          : " + j1.toString());
//bisa gausah buat toStringnya karena tadi udah buat fungsi diawal
                System.out.println("Jam 2 (String)          : " + j2.toString());
                System.out.println("Jam 1 dalam detik       : " + j1.toSeconds());
                System.out.println("Jam 2 dalam detik       : " + j2.toSeconds());

                // 4. Operasi Relational (Perbandingan)
                int perbandingan = j1.compareTo(j2);
                if (perbandingan > 0) {
```

```
                System.out.println("Status     : Jam 1 lebih lambat (setelah) Jam 2");
            } else if (perbandingan < 0) {
                System.out.println("Status     : Jam 1 lebih awal (sebelum) Jam 2");
            } else {
                System.out.println("Status     : Jam 1 dan Jam 2 sama persis");
            }

            // 5. Operasi Aritmatika
            System.out.println("Durasi (J1 ke J2)          : " + jam_2511531011.durasiDetik(j1, j2) + " detik");

            jam_2511531011 jNext = j1.nextSecond();
            System.out.println("Jam 1 detik berikutnya   : " + jNext);

            jam_2511531011 jPrev = j1.prevSecond();
            System.out.println("Jam 1 detik sebelumnya : " + jPrev);

            // 6. Operasi penjumlahan jam
            jam_2511531011 jHasilPlus = j1.plus(j2);
            System.out.println("Hasil j1 + j2                : " + jHasilPlus);

            input.close();
            System.out.println("\n Program Selesai.");

        }

            //Prosedur pembantu untuk melakukan input dan validasi secara berulang
            //sampai user memasukkan angka yang benar (0..23, 0.59).

        public static jam_2511531011 buatJamDariInput(Scanner sc) {
            int h, m, s;
            while (true) {
                System.out.print("Masukkan Jam (0-23) : ");
                h = sc.nextInt();
                System.out.print("Masukkan Menit (0-59) : ");
                m = sc.nextInt();
                System.out.print("Masukkan Detik (0-59) : ");
                s = sc.nextInt();

                // Memanggil method static isValid dari kelas jam
                if (jam_2511531011.isValid(h, m, s)) {
                    return new jam_2511531011(h, m, s);
                } else {
```

```
                                System.out.println("[Error] Input tidak valid!
Silahkan ulangi.\n");
                        }
                }
        }

}
```

Kode Program 2.9: Driver jam 2

Output program:

```
=== Program Driver Objek Jam ===

--- Input Jam 1 ---
Masukkan Jam (0-23) : 23
Masukkan Menit (0-59) : 59
Masukkan Detik (0-59) : 59

--- Input Jam 2 ---
Masukkan Jam (0-23) : 10
Masukkan Menit (0-59) : 59
Masukkan Detik (0-59) : 59

--- Hasil Operasi ---
Jam 1 (String)          : 23:59:59
Jam 2 (String)          : 10:59:59
Jam 1 dalam detik       : 86399
Jam 2 dalam detik       : 39599
Status  : Jam 1 lebih lambat (setelah) Jam 2
Durasi (J1 ke J2)       : -46800 detik
Jam 1 detik berikutnya  : 00:00:00
Jam 1 detik sebelumnya  : 23:59:58
Hasil j1 + j2           : 10:59:58

 Program Selesai.
```

Gambar 2.2: Output driver jam 2

2.4. Tugas Praktikum Pekan 1 – ADT Mobil dan Drivernya

Kode Program class Mobil_2511531011

```
package cobaMateriKuisSD;

import java.util.*;

public final class Mobil_2511531011 {
        private String nama;
        private int tahun;
        private int cc;
```

```
        private int harga;
        private String merk;

        private Mobil_2511531011[] daftarMobil;
        private int jumlah;

        public Mobil_2511531011 (String nama, int tahun, int cc, int harga, String
merk) {
                this.nama = nama;
                this.tahun = tahun;
                this.cc = cc;
                this.harga = harga;
                this.merk = merk;
        }

        public Mobil_2511531011(int kapasitas) {
                daftarMobil = new Mobil_2511531011[kapasitas];
                jumlah = 0;
        }

        public void tambahMobil(String nama, int tahun, int cc, int harga, String
merk) {
                if (jumlah < daftarMobil.length) {
                        daftarMobil[jumlah] = new Mobil_2511531011(nama,
tahun, cc, harga, merk);
                        jumlah++;
                } else {
                        System.out.println("Array Penuh!");
                }
        }

        public void hapusMobil(int index) {
                if (index >= 0 && index < jumlah) {
                        for (int i =  index; i < jumlah; i++) {
                                daftarMobil[i] = daftarMobil [i + 1];
                        }
                        jumlah --;
                        System.out.println("\nMelakukan penghapusan pada
index");
                } else {
                        System.out.println("Index tidak valid!");
                }
        }

        public String getNama() { return nama; }
        public int getTahun() { return tahun; }
        public int getCC() { return cc; }
        public int getHarga() { return harga; }
```

```
        public String getMerk() { return merk; }

        public void setNama(String nama) { this.nama = nama; }
        public void setTahun(int tahun) { this.tahun = tahun; }
        public void setCc(int cc) { this.cc = cc; }
        public void setHarga(int harga) { this.harga = harga; }
        public void setMerk(String merk) { this.merk = merk;}

        public void tampilkanMobil() {
                for (int i = 0; i < jumlah; i++) {
                        System.out.println("\nMobil ke-" + i);
                        System.out.println(daftarMobil[i].getNama() + " - " +
daftarMobil[i].getTahun() + " - " + daftarMobil[i].getCC() + " - " +
daftarMobil[i].getHarga() + " - " + daftarMobil[i].getMerk());
        }
        }

}
```

Kode Program 2.10: ADT Mobil

Kode Program class driverMobil_2511531011

```
package cobaMateriKuisSD;

public class driverMobil_2511531011 {
        public static void main(String[] args) {
                Mobil_2511531011 sistem = new Mobil_2511531011(10);

                System.out.println("\nData Mobil yang telah diinput");
                System.out.println("----------------------------");
                sistem.tambahMobil("Toyota F1", 2026, 2000, 500000000,
"Toyota");
                sistem.tambahMobil("Suzuki G1", 2025, 5000, 1000000000,
"Suzuki");
                sistem.tambahMobil("Avanza", 2020, 1300, 150000000,
"Toyota");

                sistem.tampilkanMobil();

                sistem.hapusMobil(0);

                System.out.println("\nData Mobil Saat Ini");
        System.out.println("-------------------");
                sistem.tampilkanMobil();

                System.out.println("\nSistem selesai.");
```

```
        }

}
```

Kode Program 2.11: driver mobil

Output

```
Data Mobil yang telah diinput
-----------------------------

Mobil ke-0
Toyota F1 - 2026 - 2000 - 500000000 - Toyota

Mobil ke-1
Suzuki G1 - 2025 - 5000 - 1000000000 - Suzuki

Mobil ke-2
Avanza - 2020 - 1300 - 150000000 - Toyota

Melakukan penghapusan pada index

Data Mobil Saat Ini
-------------------

Mobil ke-0
Suzuki G1 - 2025 - 5000 - 1000000000 - Suzuki

Mobil ke-1
Avanza - 2020 - 1300 - 150000000 - Toyota

Sistem selesai.
```

Gambar 2.3: Driver mobil

Penjelasan Singkat Kode Program

Pada class Mobil_2511531011 kita hanya melakukan insialisasi (ADT) untuk membuat suatu sistem. Pada sistem tersebut, kita dapat menambahkan dan mengahapus mobil sesuai dengan kriteria yang telah diinisialisi. Kita juga membuat metode selektor dan mutator untuk mempermudah kita dalam melakukan manipulasi data dan mendapatkan data untuk driver.

Pada class driverMobil_2511531011 kita membuat kode untuk menggunakan class Mobil_2511531011. Class ini memungkinkan kita untuk menambahkan data (langsung dari *source code*) dan menampilkan data menggunakan fungsi dan prosedur yang telah dibuat pada class ADT.

BAB III

KESIMPULAN

3.1. Kesimpulan

Abstract Data Type (ADT) jam dan mobil merupakan suatu bentuk implementasi konsep ADT dalam pemrograman yang digunakan untuk merepresentasikan data beserta operasi-operasi yang dapat dilakukan terhadapnya. Dengan adanya ADT, data dapat dikelola secara terstruktur melalui method yang telah didefinisikan.

Penerapan ADT dalam bahasa pemrograman Java juga membantu dalam memahami konsep pemrograman berorientasi objek, dimana data dan operasi digabungkan dalam satu kesatuan. Selain itu, penggunaan class driver mempermudah dalam menguji dan menjalankan program sehingga dapat diketahui hasil dari setiap operasi yang dilakukan.

Dengan demikian, praktikum ini memberikan pemahaman yang baik mengenai bagaimana konsep ADT dapat diterapkan dalam pembuatan program yang terstruktur dan sistematis.

3.2. Saran

Penggunaan struktur data lain seperti linked list atau array dinamis juga dapat dipertimbangkan untuk meningkatkan fleksibilitas program. Juga disarankan bagi mahasiswa untuk lebih banyak berlatih dalam membuat berbagai variasi ADT agar pemahaman terhadap konsep ini semakin mendalam. Dengan adanya pengembangan tersebut, diharapkan kemampuan dalam memahami dan mengimplementasikan struktur data dapat semakin meningkat.

DAFTAR PUSTAKA

[1] T. H. Cormen, C. E. Leiserson, R. L. Rivest, dan C. Stein, *Introduction to Algorithms*, 3rd ed., Cambridge, MA: MIT Press, 2009.

[2] Wahyudi, *Struktur Data dan Algoritma,* Padang: Informatika, 2026

[3] M.T. Goodrich, *Data Structures and Algorithms in Java*